

Python Assessment

Section A - Strings

Q1

Write a program to reverse a string without using slicing.

Example: Input: "python" Output: "nohtyp"

```
14
15     # Reverse a string
16     string = "python"
17     reversed_str = ""
18     for char in string:
19         reversed_str = char + reversed_str
20     print(reversed_str)
21
```

Run  main ×

  | 

↑ /Users/dibyamgr/PycharmProjects/PythonProject/.venv/
nohtyp
↓

Q2

Find the first non-repeating character in a string.

Example: Input: "programming" Output: "p"

```
20 # Find the first non-repeating character in a string
21 def first_non_repeating_char(s): 1 usage
22     counts = {}
23     for char in s:
24         if char in counts:
25             counts[char] += 1
26         else:
27             counts[char] = 1
28     print(counts)
29
30     for char in s:
31         if counts[char] == 1:
32             return char
33 print(first_non_repeating_char("programming"))
34
```

Run

 main ×



```
/Users/dibyamgr/PycharmProjects/PythonProject/.venv/bin/python /Us
{'p': 1, 'r': 2, 'o': 1, 'g': 2, 'a': 1, 'm': 2, 'i': 1, 'n': 1}
p
```

Q3

Check if a string is a palindrome.

Example: Input: "madam" Output: True

```
# Check if a string is a palindrome - a string that reads the same forward and backward
def is_palindrome(s): 1 usage
    # define two pointers
    left = 0
    right = len(s) - 1

    while left < right:
        if s[left] != s[right]:
            return False

        left += 1
        right -= 1

    return True

52 # Calculate the frequency of each character in a string
53 string = "hello"
54 counts = {}
55 for char in string:
56     if char in counts:
57         counts[char] += 1
58     else:
59         counts[char] = 1
60
61 print(counts)
```

Run



main



/Users/dibyamgr/PycharmProjects/PythonProject/.venv/bin/python



{'h': 1, 'e': 1, 'l': 2, 'o': 1}

Q4

Count the frequency of each character in a string.

Example: Input: "hello" Output: h:1 e:1 l:2 o:1

```
52     # Calculate the frequency of each character in a string
53     string = "hello"
54     counts = {}
55     for char in string:
56         if char in counts:
57             counts[char] += 1
58         else:
59             counts[char] = 1
60
61     print(counts)
```

Run

 main ×



```
/Users/dibyamgr/PycharmProjects/PythonProject/.venv/bin/python /Users/di
{'h': 1, 'e': 1, 'l': 2, 'o': 1}
```

Q5

Remove duplicate characters from a string while preserving order.

Example: Input: "programming" Output: "progamin"

```
63 # Remove duplicate characters from a string
64 string = "programming"
65 counts = {}
66 removed_duplicates = ""
67
68 for char in string:
69     if char not in counts:
70         counts[char] = True
71         removed_duplicates += char
72 print(removed_duplicates)
```

Run  main ×

  

↑ /Users/dibyamgr/PycharmProjects/PythonProject/.venv/bin/python /Users/d
↓ progamin

Section B - Lists

Q6

Remove duplicates from a list without using set().

Example: Input: [1,2,2,3,4,4] Output: [1,2,3,4]

```
74 # Remove duplicates from a list without using set()
75 num_list = [1,2,2,3,4,4]
76 counts = {}
77 removed_duplicates_list = []
78 for num in num_list:
79     if num not in counts:
80         counts[num] = True
81         removed_duplicates_list.append(num)
82 print(removed_duplicates_list)
83 |
```

Run

 main ×



/Users/dibyamgr/PycharmProjects/PythonProject/.venv/bin/py



[1, 2, 3, 4]

Q7

Find the second largest number in a list.

Example: Input: [10,20,5,30,25] Output: 25

```
84 # Find the second largest number in a list.
85 num_list = [10, 20, 5, 30, 25]
86
87 first_max = num_list[0]
88 second_max = num_list[0]
89
90 # first find correct first_max
91 for num in num_list:
92     if num > first_max:
93         first_max = num
94
95 # then find second max
96 for num in num_list:
97     if num != first_max and num > second_max:
98         second_max = num
99
100 print(second_max)
```

Run

 main ×



```
/Users/dibyamgr/PycharmProjects/PythonProject/.venv/bin/python /Users/dibyamgr/Pyh
25
```

Q8

Find all duplicate elements in a list.

Example: Input: [1,2,3,2,4,5,1] Output: [1,2]

```
102 # Find all duplicates in a list
103 num_list = [1,2,3,2,4,5,1]
104 count = {}
105 duplicates = []
106
107 for num in num_list:
108     if num in count:
109         count[num] += 1
110     else:
111         count[num] = 1
112
113 for num in count:
114     if count[num] > 1:
115         duplicates.append(num)
116 print(duplicates)
117
```

Run



main



/Users/dibyamgr/PycharmProjects/PythonProject/.venv/bin/python /
[1, 2]

Q9

Rotate a list by K positions.

Example: Input: [1,2,3,4,5], K=2 Output: [4,5,1,2,3]

```
117
118 # Q. Rotate a list by K positions.
119 arr = [1, 2, 3, 4, 5]
120 k = 2
121
122 n = len(arr)
123 k = k % n # reduce full rotations (n steps = same array)
124
125 # reverse whole array first
126 l = 0
127 r = n - 1
128 while l < r:
129     arr[l], arr[r] = arr[r], arr[l]
130     l += 1
131     r -= 1
132
133 print("Reversed whole array:", arr)
134 # reverse first k
135 l = 0
136 r = k - 1
137 while l < r:
138     arr[l], arr[r] = arr[r], arr[l]
139     l += 1
140     r -= 1
141
142 print("Reversed first k:", arr)
143
144 # reverse remaining
145 l = k
146 r = n - 1
147 while l < r:
148     arr[l], arr[r] = arr[r], arr[l]
149     l += 1
150     r -= 1
151
152 print("Rotated array output", arr)
```

/Users/dibyamgr/PycharmProjects/PythonProj
Reversed whole array: [5, 4, 3, 2, 1]
Reversed first k: [4, 5, 3, 2, 1]
Rotated array output [4, 5, 1, 2, 3]
Process finished with exit code 0

Q10

Find the intersection of two lists.

Example: Input: [1,2,3,4] [3,4,5,6]

Output: [3,4]

```
156 def list_intersection(lst1, lst2): 1 usage
161
162     intersection=[]
163     found_in_intersection = {}
164
165     # Step 2: Scan lst2 in O(M) time
166     for item in lst2:
167         if item in lookup:
168             if item not in found_in_intersection:
169                 intersection.append(item)
170                 found_in_intersection[item] = True
171     return intersection
172
173 print(list_intersection( lst1: [1,2,3,4], lst2: [3,4,5,6]))
174
```

Run  main ×

  

↑ /Users/dibyamgr/PycharmProjects/PythonProject/.venv/bin/python /Users/di
↓ [3, 4]

Section C - Dictionary

Q11

Count frequency of elements in a list using a dictionary.

Example: Input: [1,2,2,3,3,3] Output: {1:1, 2:2, 3:3}

```
175 # Q. Count frequency of elements in a list using a dictionary
176 def count_frequency(lst): 1 usage
177     freq = {}
178     for num in lst:
179         if num in freq:
180             freq[num] += 1
181         else:
182             freq[num] = 1
183     return freq
184
185 print(count_frequency([1,2,2,3,3,3]))
```

Run

main x



/Users/dibyamgr/PycharmProjects/PythonProject/.venv/bin/python /Users/dibyamgr/PycharmProjects/Pytho
{1: 1, 2: 2, 3: 3}

Q12

Find the key having the maximum value.

Example: {"A":100,"B":500,"C":300}

Output: B

```
189     def find_max_value(d): 1 usage
190         max_val = 0
191         max_key = ''
192
193         for key in d:
194             if d[key] > max_val:
195                 max_val = d[key]
196                 max_key = key
197         return max_key
198     print(find_max_value({'A':100,'B':500,'C':300}))
```

Run  main ×

  | 

↑ /Users/dibyamgr/PycharmProjects/PythonProject/.venv/bin/python
↓ B

Q13

Reverse a dictionary.

Example: {"a":1,"b":2}

Output: {1:"a",2:"b"}

```
200 # Q. Reverse a dictionary.
201 # Example: {"a":1,"b":2}
202 # Output: {1:"a",2:"b"}
203 def reverse_dict(d): 1 usage
204     reversed = {}
205     for key in d:
206         value = d[key]
207         reversed[value] = key
208     return reversed
209 print(reverse_dict({'a':1,'b':2}))
```

Run



main ×



```
/Users/dibyamgr/PycharmProjects/PythonProject/.venv/bin/python /
{1: 'a', 2: 'b'}
```

Q14

Merge two dictionaries.

Example: d1={"a":1} d2={"b":2}

Output: {"a":1,"b":2}

```
211 # Merge two dictionaries.
212 # Example: d1={"a":1} d2={"b":2}
213 # Output: {"a":1,"b":2}
214 def merge_dict(d1, d2): 1 usage
215     merged = {}
216     for key in d1:
217         merged[key] = d1[key]
218     for key in d2:
219         merged[key] = d2[key]
220      return merged
221 print(merge_dict(d1: {'a': 1},| d2: {'b': 2}))
```

Run

 main ×



/Users/dibyamgr/PycharmProjects/PythonProject/.venv/bin/python /Users/d
{'a': 1, 'b': 2}

Q15

Count word frequency in a sentence using dictionary.

Example: Input: "python is good python is easy"

Output: { "python":2, "is":2, "good":1, "easy":1 }

```
224 # Example: Input: "python is good python is easy"
225 # Output: { "python":2, "is":2, "good":1, "easy":1 }
226 def count_word_freq(sentence): 1 usage
227     words = []
228     current_word = ""
229     for char in sentence:
230         if char == " ":
231             if current_word != "":
232                 words.append(current_word)
233                 current_word = ""
234             else:
235                 current_word += char
236         if current_word != "":
237             words.append(current_word)
238
239     freq = {}
240     for word in words:
241         if word in freq:
242             freq[word] += 1
243         else:
244             freq[word] = 1
245     return freq
246
247 print(count_word_freq("python is good python is easy"))
```

Run

main x



```
/Users/dibyamgr/PycharmProjects/PythonProject/.venv/bin/python /Users/dibyamgr/PycharmProje
{'python': 2, 'is': 2, 'good': 1, 'easy': 1}
```

Section D - Loops

Q16

Print the following pattern:

• **

```
249 # Q.Print the following pattern:
250 #    **
251 rows = 1
252 cols = 2
253
254 for i in range(rows):
255     for j in range(cols):
256         print("*", end="")
257     print()
```

Run

 main ×



/Users/dibyamgr/PycharmProjects/PythonProject/.venv/bin/python /Users/di

**

Q17

Print multiplication table of a given number.

Example: Input: 5

Output: 5 x 1 = 5 ... 5 x 10 = 50

```
259 # Q. Multiplication Table
260 n=5
261 for i in range(1, 11):
262     print(str(n) + " x " + str(i) + " = " + str(n * i))
```

Run  main x

  

↑	5 x 1 = 5
↓	5 x 2 = 10
↔	5 x 3 = 15
↔	5 x 4 = 20
↔	5 x 5 = 25
↔	5 x 6 = 30
↔	5 x 7 = 35
↔	5 x 8 = 40
↔	5 x 9 = 45
↔	5 x 10 = 50

Q18

Find factorial using for loop.

Input: 5 Output: 120

```
264     # Factorial
265     n = 5
266     result = 1
267     for i in range(1, n + 1):
268         result *= i
269     print(result)
```

Run



main ×



/Users/dibyamgr/PycharmProjects/PythonProject/.ven
120

Q19

Find all prime numbers between 1 and 100.

```
271 # Prime Numbers
272 def find_primes(): 1 usage
273     primes = []
274     for num in range(2, 101):
275         is_prime = True
276
277         # Optimization: We only need to check up to i * i <= num
278         i = 2
279         while i * i <= num:
280             if num % i == 0:
281                 is_prime = False
282                 break
283             i += 1
284
285         if is_prime:
286             primes.append(num)
287     return primes
288
289 print(find_primes())
```

Run main ×

↑ /Users/dibyamgr/PycharmProjects/PythonProject/.venv/bin/python /Users/dibyamgr/PycharmProjects/PythonProje
↓ [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]

Q20

Generate Fibonacci series up to N terms.

Example: Input: 8

Output: 0 1 1 2 3 5 8 13

```
291 # Fibonacci series
292 def fibonacci(n): 1 usage
293     if n <= 0:
294         return []
295     if n == 1:
296         return [0]
297
298     series = [0, 1]
299     for i in range(2, n):
300         next_term = series[i - 1] + series[i - 2]
301         series.append(next_term)
302     return series
303
304 print(fibonacci(8))
```

Run

 main ×



```
/Users/dibyamgr/PycharmProjects/PythonProject/.venv/bin/python /Users/di
[0, 1, 1, 2, 3, 5, 8, 13]
```

Section E - Interview Coding Questions

Q21

Find the first non-repeating number in a list.

Input: [1,2,3,4,5,1,2,3]

Output: 4

```
307 # Find the first non-repeating number in a list.
308 # Input: [1,2,3,4,5,1,2,3]
... 309 def first_non_repeating_num(lst): 1 usage
310     counts = {}
311     for num in lst:
312         if num in counts:
313             counts[num] += 1
314         else:
315             counts[num] = 1
316
317     for num in lst:
318         if counts[num] == 1:
319             return num
320     return None
321
322 print(first_non_repeating_num([1,2,3,4,5,1,2,3]))
```

Run  main x

  

↑ /Users/dibyamgr/PycharmProjects/PythonProject/.venv/bin/python /Users/dibyamgr/Pych
↓ 4

Q22

Find the Nth non-repeating number in a list.

Input: [1,2,3,4,5,1,2,3] N = 2

Output: 5

```
324 # Q22
325 # Find the Nth non-repeating number in a list.
326 # Input: [1,2,3,4,5,1,2,3] N = 2
327
328 def nth_non_repeating_num(lst, n): 1 usage
329     counts = {}
330     for num in lst:
331         if num in counts:
332             counts[num] += 1
333         else:
334             counts[num] = 1
335
336     non_repeating = []
337     for num in lst:
338         if counts[num] == 1:
339             non_repeating.append(num)
340
341     unique_count = 0
342     for _ in non_repeating:
343         unique_count += 1
344
345     if unique_count >= n:
346         return non_repeating[n - 1]
347     return None
348
349 print(nth_non_repeating_num(lst=[1,2,3,4,5,1,2,3], n=2))
```

Run

main x



5

/Users/dibyamgr/PycharmProjects/PythonProject/.venv/bin/python /Users/dibyamgr/PycharmProject

5

Q23

Check whether two strings are anagrams.

Input: "listen" "silent"

Output: True

```
352 def are_anagrams(s1, s2): 1 usage
353     # O(N) frequency calculation
354     counts1 = {}
355     for char in s1:
356         if char in counts1:
357             counts1[char] += 1
358         else:
359             counts1[char] = 1
360
361     # O(M) frequency calculation
362     counts2 = {}
363     >  for char in s2: ...
368
369     # O(1) character validation dictionary checks
370     for key in counts1:
371         if key not in counts2 or counts1[key] != counts2[key]:
372             return False
373
374     for key in counts2:
375         if key not in counts1:
376             return False
377
378     return True
379
380 print(are_anagrams(s1: 'listen', s2: 'silent'))
```

Run  main x

  | :

↑ /Users/dibyamgr/PycharmProjects/PythonProject/.venv/bin/python /Users/dibya
↓ True

Q24

Find missing number from array.

Input: [1,2,3,5]

Output: 4

```
382     def find_missing_number(lst): 1 usage
383         max_val = 0
384         for num in lst:
385             if num > max_val:
386                 max_val = num
387
388         # expected sum formula
389         expected_sum = (max_val * (max_val + 1)) // 2
390
391         # actual sum pass
392         actual_sum = 0
393          for num in lst:
394             actual_sum += num
395
396         return expected_sum - actual_sum
397
398     print(find_missing_number([1,2,3,5]))
```

Run

 main x



/Users/dibyamgr/PycharmProjects/PythonProject/.venv/bin/python /Users/



4

Q25

Find top occurring element in a list.

Input: [1,2,2,3,3,3,4]

Output: 3

```
400 # Q25
401 # Find top occurring element in a list.
402 # Input: [1,2,2,3,3,3,4]
403 # Output: 3
404 def top_occurring(lst): 1 usage
405     counts = {}
406     for item in lst:
407         if item in counts:
408             counts[item] += 1
409         else:
410             counts[item] = 1
411
412     max_key = None
413     max_val = 0
414     for key in counts:
415         if counts[key] > max_val:
416             max_val = counts[key]
417             max_key = key
418
419     return max_key
420
421 print(top_occurring([1,2,2,3,3,3,4]))
```

Run

main ×



/Users/dibyamgr/PycharmProjects/PythonProject/.venv/bin/python /Users/dibyamgr/PycharmP

3



Bonus: Explain the difference between:

- List vs Tuple

List	Tuple
Mutable (can add, remove, or change items)	Immutable (cannot be altered after creation)
Larger overhead to allow dynamic resizing	Smaller footprint; fixed allocation size
Slower processing and iteration speeds	Faster read performance and iteration
Ideal for collections of data that will change.	Used for fixed configurations or as dictionary keys.

- Dictionary vs Set

Dictionary	Set
Stores data as paired Key-Value associations	Stores unique individual elements only
Elements are retrieved instantly via their unique key	Checked via element presence (no indexed keys)
Keys must be unique; values can duplicate.	Completely filters out duplicate items.

- Shallow Copy vs Deep Copy

Shallow copy	Deep copy
Creates a brand new top-level container object.	Creates a brand new top-level container object
Copies references to nested data structures	Recursively duplicates nested data elements
Changing nested items modifies the original object.	Entirely isolated; changes never impact the original.

- **is vs ==**

== (Equality): Checks if the values of two objects are equal. It looks at what is inside the data structures

Example: `[1, 2] == [1, 2]` yields `True`.

is (Identity): Checks if two variables point to the exact same object in memory. It compares the internal memory addresses (`id(a) == id(b)`).

Example: `[1, 2] is [1, 2]` yields `False` (distinct lists).

- **append() vs extend()**

append(item):

- Accepts any object (integers, strings, or whole lists)
- Pushes the argument down as one single element
- Increases the overall list length by exactly +1.

extend(iterable):

- Expects an iterable sequence (lists, tuples, strings).
- Iterates over the items and adds each one individually.
- Increases length by the number of elements added.